

Topic 16: ArrayList

What is an ArrayList?

You already know arrays. An array is fixed in size — once you create it with 5 slots, it always has 5 slots. You can't add a 6th element without creating a brand new array.

ArrayList solves this. It's a **resizable, dynamic list** that grows and shrinks automatically as you add or remove elements.

Feature	Array	ArrayList
Size	Fixed	Dynamic (auto-resize)
Syntax	<code>int[]</code>	<code>ArrayList<Integer></code>
Built-in methods	None	add, remove, contains, sort...
Stores primitives?	 Yes	 No (uses wrapper types)
Part of	Core Java	<code>java.util</code> package

Setting Up — Import

ArrayList lives in the `java.util` package. Always import it first:

```
java
import java.util.ArrayList;
```

Creating an ArrayList

```
java
import java.util.ArrayList;

ArrayList<String> names = new ArrayList<>();
ArrayList<Integer> scores = new ArrayList<>();
ArrayList<Double> prices = new ArrayList<>();
```

The type inside `<>` is called a **type parameter** — it tells the ArrayList what kind of data it holds.



You cannot use primitive types directly. Use their **wrapper classes** instead:

Primitive

Wrapper (use in ArrayList)

int

Integer

double

Double

char

Character

boolean

Boolean

add() — Adding Elements

Add to the end:

```
java
ArrayList<String> fruits = new ArrayList<>();

fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");

System.out.println(fruits); // [Apple, Banana, Mango]
```

Add at a specific index:

```
java

fruits.add(1, "Orange"); // inserts at index 1, shifts others right
System.out.println(fruits); // [Apple, Orange, Banana, Mango]
```

get() — Accessing Elements

```
ArrayList<String> fruits = new ArrayList<>();
fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");

System.out.println(fruits.get(0)); // Apple
System.out.println(fruits.get(2)); // Mango
```

Just like arrays, indexing starts at **0**.

remove() — Removing Elements

Remove by index:

```
java

fruits.remove(1); // removes "Banana" (index 1)
System.out.println(fruits); // [Apple, Mango]
```

Remove by value:

```
java

fruits.remove("Mango"); // removes the element "Mango"
System.out.println(fruits); // [Apple]
```

⚠ Gotcha with integers: When you call `remove(2)` on an `ArrayList<Integer>`, Java treats `2` as an **index**, not a value. To remove the value `2`, use:

```
java

scores.remove(Integer.valueOf(2));
```

size() — Number of Elements

```
java

ArrayList<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Charlie");

System.out.println(names.size()); // 3
```

Other Essential Methods

```
java

ArrayList<String> list = new ArrayList<>();
list.add("Java");
list.add("Python");
list.add("C++");
list.add("Java");

// Check if an element exists
System.out.println(list.contains("Python")); // true
System.out.println(list.contains("Ruby")); // false

// Find index of an element
System.out.println(list.indexOf("Java")); // 0 (first occurrence)
System.out.println(list.lastIndexOf("Java")); // 3

// Check if empty
System.out.println(list.isEmpty()); // false

// Replace element at index
list.set(1, "JavaScript");
System.out.println(list); // [Java, JavaScript, C++, Java]

// Clear all elements
list.clear();
System.out.println(list.isEmpty()); // true
```

Iteration — Looping Through an ArrayList

Method 1 — for loop with index:

```
java

ArrayList<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Charlie");

for (int i = 0; i < names.size(); i++) {
    System.out.println(i + ": " + names.get(i));
}
```

Method 2 — enhanced for loop (for-each):

```
java

for (String name : names) {
    System.out.println(name);
}
```

Method 3 — forEach with lambda (modern style):

```
java

names.forEach(name -> System.out.println(name));
// or even shorter:
names.forEach(System.out::println);
```

Dynamic Resizing — How It Works Internally

When you create an `ArrayList`, it starts with an internal array of capacity **10**. When you add the 11th element, Java automatically:

1. Creates a new array with **1.5× the capacity** (15 slots)
2. Copies all old elements into it
3. Adds the new element

This is all handled **automatically** — you never have to worry about it. That's the whole point of `ArrayList`.

```
java

ArrayList<Integer> numbers = new ArrayList<>();

for (int i = 1; i <= 20; i++) {
    numbers.add(i); // grows automatically beyond 10
}

System.out.println("Size: " + numbers.size()); // 20
System.out.println(numbers);
// [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

Sorting an ArrayList

```
java
```

```
import java.util.ArrayList;
import java.util.Collections;

ArrayList<Integer> scores = new ArrayList<>();
scores.add(45);
scores.add(92);
scores.add(67);
scores.add(13);
scores.add(88);

Collections.sort(scores); // ascending
System.out.println(scores); // [13, 45, 67, 88, 92]

Collections.sort(scores, Collections.reverseOrder()); // descending
System.out.println(scores); // [92, 88, 67, 45, 13]
```

ArrayList vs Array — When to Use Which?

Use **Array** when:

- You know the exact size and it won't change
- You're working with primitives and performance is critical
- e.g., storing 12 months of data, a fixed-size board game grid

Use **ArrayList** when:

- The number of elements changes at runtime
- You need built-in methods like search, sort, remove
- e.g., a shopping cart, a list of students, search results

Full Example 1: Student Name Manager

```
java
```

```
import java.util.ArrayList;
import java.util.Scanner;

public class StudentManager {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();
        Scanner scanner = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n=== Student Manager ===");
            System.out.println("1. Add student");
            System.out.println("2. Remove student");
            System.out.println("3. View all students");
            System.out.println("4. Search student");
            System.out.println("5. Exit");
            System.out.print("Choice: ");
            choice = scanner.nextInt();
            scanner.nextLine(); // consume newline

            switch (choice) {
                case 1:
                    System.out.print("Enter name: ");
                    String name = scanner.nextLine();
                    students.add(name);
                    System.out.println(name + " added.");
                    break;

                case 2:
                    System.out.print("Enter name to remove: ");
                    String removeName = scanner.nextLine();
                    if (students.remove(removeName)) {
                        System.out.println(removeName + " removed.");
                    } else {
                        System.out.println("Student not found.");
                    }
                    break;

                case 3:
                    if (students.isEmpty()) {
                        System.out.println("No students yet.");
                    } else {
                        System.out.println("Students (" + students.size() + "):");
                        for (int i = 0; i < students.size(); i++) {
                            System.out.println((i + 1) + ". " + students.get(i));
                        }
                    }
                }
            }
        } while (choice != 5);
    }
}
```

```
    }  
    break;  
  
    case 4:  
        System.out.print("Search name: ");  
        String search = scanner.nextLine();  
        System.out.println(students.contains(search) ? "Found!" : "Not found.");  
        break;  
  
    case 5:  
        System.out.println("Goodbye!");  
        break;  
    }  
  
    } while (choice != 5);  
    }  
}
```

Full Example 2: Score Tracker

```
java
```



```

import java.util.ArrayList;
import java.util.Collections;

public class ScoreTracker {
    public static void main(String[] args) {
        ArrayList<Integer> scores = new ArrayList<>();
        scores.add(78);
        scores.add(92);
        scores.add(55);
        scores.add(88);
        scores.add(40);
        scores.add(95);

        System.out.println("All Scores: " + scores);
        System.out.println("Total Students: " + scores.size());

        // Sum and average
        int sum = 0;
        for (int s : scores) sum += s;
        System.out.printf("Average: %.2f%n", (double) sum / scores.size());

        // Min and Max
        System.out.println("Highest: " + Collections.max(scores));
        System.out.println("Lowest : " + Collections.min(scores));

        // Sorted
        Collections.sort(scores);
        System.out.println("Sorted : " + scores);

        // Remove failing scores (below 50)
        scores.removeIf(s -> s < 50);
        System.out.println("Passing scores only: " + scores);
    }
}

```

Common Mistakes & How to Avoid Them

✗ Using primitive type in ArrayList:

```

java
ArrayList<int> numbers = new ArrayList<>(); // ERROR
ArrayList<Integer> numbers = new ArrayList<>(); // ✓ Correct

```

✗ Using == to compare String elements:

```
java

if (list.get(0) == "Alice") { ... }      // unreliable
if (list.get(0).equals("Alice")) { ... } // ✓ Correct
```

✗ Modifying a list while iterating with for-each:

```
java

for (String item : list) {
    list.remove(item); // ✗ throws ConcurrentModificationException
}

// ✓ Use removeIf instead:
list.removeIf(item -> item.equals("Alice"));
```

✗ Forgetting remove(index) vs remove(value) for integers:

```
java

ArrayList<Integer> nums = new ArrayList<>();
nums.add(10);
nums.add(20);
nums.add(30);

nums.remove(1);           // removes index 1 → removes 20
nums.remove(Integer.valueOf(10)); // removes the value 10 ✓
```

Quick Reference Card

Method	What it does
<code>add(value)</code>	Adds to the end
<code>add(index, value)</code>	Inserts at position
<code>get(index)</code>	Returns element at index
<code>set(index, value)</code>	Replaces element at index
<code>remove(index)</code>	Removes by position
<code>remove(Object)</code>	Removes by value

Method	What it does
<code>size()</code>	Returns number of elements
<code>contains(value)</code>	Returns true/false
<code>indexOf(value)</code>	Returns first index of value
<code>isEmpty()</code>	Returns true if no elements
<code>clear()</code>	Removes all elements
<code>Collections.sort(list)</code>	Sorts ascending
<code>Collections.max(list)</code>	Returns largest value
<code>Collections.min(list)</code>	Returns smallest value

Practice Exercises

1. Create an `ArrayList` of 5 city names. Sort them alphabetically and print.
2. Take 10 integers as input from the user into an `ArrayList`. Remove all even numbers and print what's left.
3. Write a program that maintains a shopping cart — user can add items, remove items, and view the cart.
4. Given an `ArrayList` of marks, count how many students passed (marks ≥ 50) and how many failed.
5. Create two `ArrayLists` of integers. Merge them into a third `ArrayList`, remove duplicates, sort it, and print.

What's Next?

You now know how to work with dynamic lists. In the next topics you'll explore the full **Java Collections Framework** — which includes `Set` (no duplicates), `Map` (key-value pairs), and more powerful data structures built for real-world programs.